



Technology Stack Recommendation for Ko'Ride

Context and challenges of the Ko'Ride project

Ko'Ride is an upcoming ridesharing app targeting the Indonesian market. The development team is junior (4 members) with varied skills: David (beginner/intermediate backend), Jean-Charles (junior fullstack), Yann (Angular frontend), and Léo (UX/UI). The context imposes several constraints: the solution must be efficient, quick to develop, low cost, and usable by local developers in Indonesia. In addition, the backend will have to be centralized to power both a mobile application and a website (via a REST API or GraphQL). The chosen technology stack must not only allow for the rapid construction of an MVP, but also be able to scale to a large-scale production application.

The team already has initial preferences: - **Web frontend:** Angular (framework mastered by Yann). - **Backend:** Java Spring Boot, or a Node.js solution (Express or NestJS in TypeScript). - **Mobile:** No specific mobile experience at this stage, but open to React Native, Flutter or other accessible cross-platform solutions.

The aim is therefore to propose a realistic and progressive technical architecture, taking advantage of the best practices observed among international carpooling leaders (BlaBlaCar, Grab, Lyft, Uber) while adapting it to a small team and the local Indonesian context.

Lessons from the stacks used by BlaBlaCar, Grab, Lyft, Uber

Major shared mobility applications have evolved their stacks over time to achieve performance and scalability. It's worth learning a few lessons before choosing Ko'Ride's stack:

- **Monolithic vs. Microservices Architecture:** Many of these companies started with a simple monolithic system and then moved to a microservices architecture as they grew. For example, Uber started with a monolithic backend written in Python and a single PostgreSQL database.

1 This choice was sufficient during the initial phase, until the scaling up and expansion of the development team made it necessary to transition to microservices (in order to better segment functionality and improve scale)

1 2 . Similarly, BlaBlaCar later adopted a microservices architecture to gain scalability and allow independent deployment of features. Approaches show that a **well-designed monolith can support an MVP** 3 . These **and the beginnings**, and that **migration to microservices** can come later when needs dictate.

4 It's important to note that moving to microservices too early adds unnecessary complexity for a small team – conventional wisdom recommends starting monolithic and then scaling up once the complexity is proven.

- **Backend languages and frameworks used:** The giants' stacks are often polyglot. Uber, for example, has over time leveraged **multiple backend languages** to serve its

microservices: Python (Flask) historically, Node.js for its initial dispatch engine, then Go and Java for various services. Uber says its trip calculation engine had been originally written in Node.js to take advantage of its asynchronous model and ability to handle a large number of concurrent connections, before introducing Go for reasons of performance and type safety. Lyft has adopted primarily an architecture of microservices in Python (Flask) on the server side, with web frontends in AngularJS then React according to the services.⁷ Grab, for its part, has evolved its stack towards microservices in Go (via their internal Grab-Kit framework) to take advantage of Go's concurrency and efficiency.⁸ BlaBlaCar used a mixed stack including Node.js and Go, and turned to Google Cloud Platform to benefit from a scalable infrastructure capable of responding to real-time to the needs of its global community. These examples⁹ suggest that there is no single "magic language": the choices depend on the stage of development and expertise available. For Ko'Ride, this means choosing a backend that the team is familiar with and that allows rapid prototyping, while being efficient enough to evolve.

- **Front-end and mobile stacks of the major players:** Leaders focused on experience native user. Uber and Lyft have developed native mobile applications (Swift/Objective-C for iOS, Java/Kotlin for Android) to optimize performance and deep integration (GPS, notifications, etc.). BlaBlaCar also offers native apps. However, Developing two apps natively requires two separate teams, which is not realistic for a small team. **Modern cross-platform solutions did not exist at launch of these companies**, but today frameworks like **React Native** or **Flutter** offer an interesting compromise: a single mobile code for Android and iOS, with performance close to the native. Moreover, many "clones" or new carpooling players are opting for cross-platform technologies to get started faster. For example, a guide to carpooling development notes that common stacks include *React Native* for mobile cross-platform, coupled with a Node.js backend and sometimes Firebase for functionalities in the cloud.^{10,11} This shows that a carpooling startup can absolutely succeed by starting on cross-platform for mobile and a Node backend, without immediately matching the complex infrastructure of an Uber.
- **Databases and Data Scalability:** Initially, a relational database single instance can be enough. Uber managed with a single PostgreSQL instance during its infancy.² the increase in traffic, Uber moved to a more segmented approach to data: they have migrated some data to sharded MySQL (internal "Schemaless" solution), and used NoSQL databases (Riak, Cassandra) for very large scale or real-time needs, while implementing Redis caching. BlaBlaCar, as it grew,^{12,13} had to migrate from a on-premises hosting to the cloud (GCP) and adopting Kubernetes to manage more efficiently load and international growth. **The lesson for Ko'Ride**¹⁴ is that a standard SQL database (PostgreSQL or MySQL) can handle the MVP application data, but it will have to be architected to evolve (for example by providing geospatial indexes for search for journeys by location, being able to partition the data by region if necessary, etc.). We also note that the addition of **in-memory caches** (e.g. Redis) and possibly databases Specific NoSQL can be done later to optimize performance when the audience explodes. In short, start simple, then adopt distributed storage solutions at as it becomes necessary, is a path proven by these companies.

In summary, the big names in the sector teach us to **prioritize simplicity and speed. development at the start**, while keeping in mind a possible evolution towards a more modular. It is necessary to adopt tools that are known and masterable by the current team and local recruits

potential, and design the system in a modular way to be able to scale it out when the volume of users requires it.

Web frontend: Angular for the website

Given the team's expertise in Angular, it makes sense to leverage it for Ko'Ride's front-end. Angular is a robust and comprehensive front-end framework, well-suited for building a feature-rich web application. It will enable the development of a web interface for Ko'Ride where users can search for rides, manage their profile, publish trips, and more. The advantages of Angular in this context are:

- **Productivity and structure:** Angular provides an MVC/modular structure that helps junior developers organize their code into components, services, modules, and so on. This is consistent with good maintainability practices. Furthermore, since Jean-Charles and Yann are already familiar with it, development will be faster than with a technology that's new to them.
- **Ecosystem and tools:** Angular has a mature ecosystem (CLI to scaffold the project, numerous UI components, form binding, etc.). Existing libraries can be used to integrate, for example, Google Maps into the page (via Angular Google Maps), manage authentication via JWT, etc., which accelerates development.
- **TypeScript:** Angular uses TypeScript, which provides type safety that helps reduce bugs while remaining within the familiar JavaScript universe. This TS/JS consistency can be an asset if the backend also uses TypeScript (more on this later).
- **Client-side performance:** Angular, although heavier than frameworks like React in terms of initial bundle, offers good runtime performance and lazy-loading tools that will be useful for loading, for example, the map only when necessary. For the Indonesian market, it will be necessary to optimize the application for sometimes limited mobile connections (e.g., by managing a PWA *Progressive Web App* so that the site can be viewed offline or with a weak network, and by minimizing the weight of the bundles).
- **Community and Recruitment:** Angular is used worldwide, including in Southeast Asia, although React is also very popular there. It's expected that you'll be able to recruit Angular developers or train Indonesian front-end developers on Angular relatively quickly, given that it's a well-documented and widely taught framework.

Additionally, it would be wise to implement a **responsive** and mobile-friendly design on the Angular application from the start, so that the website can serve as a fallback solution for users who do not install the mobile application. A PWA (Progressive Web App) could be considered – Angular supports it – but given the nature of carpooling (push notifications, continuous geolocation, deep GPS integration), **a native or cross-platform mobile application will be necessary** for the optimal user experience.

Mobile application: favor a cross-platform solution

The Indonesian market is very mobile-oriented – Android dominates, with a significant iOS base among professionals and middle classes.¹⁵ To reach as many people as possible, Ko'Ride will need to offer a mobile app on **Android and iOS**. Given the small size of the team and the lack of native mobile expertise in-house, the choice of a **cross-platform framework**

is a must for the MVP. The two main modern options are **React Native** and **Flutter**, to which we can add Ionic (Angular) as an outsider. Let's analyze these options:

- **React Native (RN):** RN allows you to develop in JavaScript/TypeScript, using the React framework. Advantage for the team: the language would be the same as Angular's (TypeScript), and Jean-Charles, in fullstack JS, could more easily learn React Native. The RN community is large, many modules exist (for Google Maps, for notifications, etc.), which speeds up development. The disadvantage is that you would have to learn the React philosophy (based on components, hooks, etc.), which is different from Angular's. Nevertheless, RN is proven: many popular apps use it. It offers near-native performance thanks to the native rendering of UI components. RN could therefore be a good choice if the team feels ready to learn React.
- **Flutter :** Flutter, developed by Google, uses the Dart language. It is a cross-platform framework that has the advantage of **providing near-native performance** (Dart code is compiled natively) and offers very high development speed thanks to *hot-reload* and a rich set of pre-designed widgets. Flutter is particularly appreciated for its UI consistency between Android and iOS – you code a single interface that will render identically on both platforms. For Ko'Ride, Flutter has several advantages: fast for building complex interfaces (e.g., map display, smooth animations), good support for features like geolocation (Google Maps plugins, etc.), and **a growing ecosystem in Indonesia**. Indeed, Flutter is gaining popularity in the region, and it may be easier to recruit young local developers familiar with Flutter/Dart, as the learning curve is relatively gentle and Google promotes Flutter in dev communities. The main obstacle is that no one on the current team knows Dart/Flutter, so they'll have to learn a new framework. However, since the entire team is starting from scratch in mobile, learning Flutter or RN is an equivalent effort, with perhaps the advantage of very didactic Flutter documentation and a unified codebase.
- **Ionic / Angular + Capacitor:** This option would consist of reusing Angular skills to create a hybrid application. Ionic allows you to develop a mobile application in Angular (or React/Vue) and deploy it in a native WebView via Capacitor or Cordova, with access to native APIs. The advantage is the maximum reuse of Angular (so familiar territory for the team) and a single web+mobile code base. However, the disadvantages are notable for a case like carpooling: performance is generally lower (encapsulated web application, less fluid than a true native or Flutter app, especially for animations or the map), and limitations on access to advanced features (background GPS, very reactive notifications) can be problematic or require plugins that are sometimes less robust. Ionic could be useful for a quick prototype, but if Ko'Ride is aiming for an application quality on the level of Grab or BlaBlaCar, it is better to move towards RN or Flutter which give a **100% native** rendering in the end.

Considering these points, **our recommendation leans towards Flutter** as a cross-platform mobile solution. Flutter would offer high development speed and a native-quality user experience on both leading platforms. Moreover, choosing Flutter would be consistent with the fact that Angular is already used on the web, since Google maintains both technologies (the component/widget concepts have some similarities in the declarative UI approach). With Flutter being increasingly adopted, the team will eventually be able to recruit Flutter developers in Indonesia quite easily, where the community is growing. Nevertheless, if the team feels more comfortable with the JavaScript ecosystem, **React Native remains an excellent choice**, supported by a large community, and

could leverage existing TypeScript know-how. In both cases, the benefit is to avoid having two code bases (Android + iOS) to maintain.

In terms of key mobile features to implement, we will need: - **Geolocation & Mapping** : Google Maps or Mapbox integration to display journeys, locate drivers/passengers on the map, calculate routes. Both Flutter and RN have ready libraries (e.g. `google_maps_flutter` for Flutter, or `react-native-maps` for RN). - **Push Notifications** : Essential for alerting of new messages, carpool confirmations, etc. We will use Firebase Cloud Messaging (FCM) for Android and APNS for iOS – often managed via Firebase to simplify (Flutter and RN have plugins for FCM).

20 - **Local storage & offline mode**: Provide a local cache (e.g. embedded SQLite or Hive local database under Flutter) to keep the history of journeys or messages in offline mode. - **Performance and optimization**: Take into account that many users will have entry/mid-range Android smartphones.

Flutter has the advantage of rendering very well even on modest devices, and RN requires optimizing work on the JavaScript thread to avoid blocking the UI. In any case, it will be necessary to test on various terminals.

In conclusion, **developing the Ko'Ride app cross-platform** will allow for rapid delivery on Android and iOS without doubling costs. Flutter is recommended for its speed of development and consistent UI, but React Native would be a perfectly valid choice if the team prefers to stay within the JS ecosystem. In any case, the Angular web app will continue to exist in parallel for desktop users or for web SEO, and possibly as a PWA.

rescue.

Centralized backend: unified API for web and mobile

Ko'Ride's backend will be the core, exposing APIs consumed by the mobile app and website. This backend will handle user accounts, trip creation and search, booking, payments, internal messaging, etc. It will also support real-time features (e.g., driver location updates, instant notifications). Here are our recommendations for the backend stack, taking into account the team's preferences (Java Spring Boot vs. Node.js/NestJS) and the project's requirements:

- **Backend language and framework**: Given the experience of David (backend) and Jean-Charles (fullstack) who seem web-oriented, a natural fit is **Node.js with a TypeScript framework**. In particular, **NestJS** could be a great choice, as it draws heavily from the Angular architecture (decorators, dependency injection, modular structure) which would make it easier for the team to get started. NestJS runs on Node.js, benefiting from its event-driven and non-blocking model, ideal for real-time applications with many concurrent connections. This fits well with the needs of a ride-sharing app where you might have multiple users refreshing the trip list or sending messages. In fact, Uber has shown that a critical service written in Node.js can handle a large number of concurrent connections thanks to asynchronization.

6 Additionally, a Node/TypeScript backend would allow for front-back **language consistency** (all in TS), which simplifies development for a small team (developers can more easily navigate between the Angular front and the NestJS back).

Alternatively, **Java Spring Boot** is a powerful enterprise-grade framework that can be used to build a robust REST API. Spring Boot offers *batteries-included features* (security, DB access, etc.) and high multi-threaded performance. However, for a junior team, Spring Boot might have a steeper learning curve (configuration, concepts like Beans, etc.), especially

without an experienced Java mentor. Node.js/NestJS would be lighter in terms of cognitive load and could speed up prototyping. Moreover, from a **recruitment perspective in Indonesia**, there are many Java developers in the sector (thanks to Android and J2EE companies), but there is also a whole new generation comfortable with JavaScript/Node.js. Since Node.js is very popular for startups and web projects, it might be easier to find motivated junior Node/JS profiles to join Ko'Ride.

Overall, we recommend **NestJS (Node.js + TypeScript)** for the MVP backend. This doesn't preclude the possibility of integrating Spring Boot or another language later for specific microservices if a specific need arises (e.g., a very advanced route calculation module that could benefit from Java/Go), but starting with Node/Nest offers the best speed/convenience compromise.

- **API: REST or GraphQL?** To serve both the Angular web frontend and the mobile app, you'll need to design APIs. The choice between REST and GraphQL depends on your needs and skills.
REST is the simplest and most universal choice: the team can quickly create RESTful controllers in NestJS or Spring Boot exposing JSON endpoints. It's widely understood by all developers (easier to hand off to junior recruits) and well supported (e.g., Angular has an HttpClient to consume REST). **GraphQL** offers more flexibility on the client side (you can request exactly the fields you want, avoid multiple nested queries), which can optimize network exchanges for mobile. Companies like Facebook or Yelp use GraphQL, and we're seeing GraphQL APIs appearing in mobility services, but this isn't (to our knowledge) the standard at Uber/Grab for consumer mobile apps—Uber instead favors gRPC or internally optimized REST calls. For Ko'Ride, **we suggest starting with REST**, which is easier to implement. NestJS could potentially support GraphQL later via a dedicated module, if there is a benefit (e.g., to unify profile + journey + message data in one call that would otherwise be provided by multiple REST endpoints). But at the MVP stage, the clarity of a well-designed REST API will be an asset.

Each main resource (users, journeys, bookings, payments, messages) will have its own set of CRUD endpoints and specific actions. These endpoints will need to be secured (authentication via JWT token, see security section).

- **Real-time management (notifications, live geolocation):** A crucial point for Ko'Ride will be the management of *live features*: tracking a driver's position on the map in real time, instantly receiving a message or booking confirmation. Two approaches can be combined:
 - **WebSockets:** With Node.js, you can use Socket.io or the native WebSocket library to push real-time information from the server to clients. When a driver updates their position or a new chat message is ²² posted, the backend sends the update to the relevant clients via a WebSocket channel rather than waiting for them to make a request. NestJS integrates Socket.io well. This is suitable for position tracking with updates every X seconds, and for instant chat.
 - **Firebase (BaaS):** An alternative or complement is to delegate some real-time aspects to managed services. For example, **Firebase Realtime Database** or **Firestore** can be used to stream live location or chat messages – the Flutter/RN client can listen for changes in the database in real time without managing the WebSocket server infrastructure. Similarly, **Firebase Cloud Messaging (FCM)** will be used anyway for system push notifications (because on mobile, we have to go through FCM/APNS). The team could choose to use Firebase to speed up the development of the MVP (especially if backend skills are limited).

For example, storing chat conversations in Firestore would greatly simplify implementation instead of messaging (Firebase takes care of real-time synchronization, scalability, etc.)

23 22 .

However, relying exclusively on Firebase can limit development freedom on the long term and generates variable costs. A compromise may be interesting: **use**

Firestore for push notifications and possibly real-time chat, while

keeping the main backend (Node) for business logic, journey persistence and reservations, etc. This is similar to what some carpooling tutorials offer: an architecture hybrid where *“our own server handles the core logic, and Firebase serves for real-time streams specific”*

23 .

- **Main database:** We recommend using a **relational** database

SQL as the foundation, particularly **PostgreSQL** (or MySQL as an alternative). These DBMSs are open-source, reliable, and well-established in the industry, including for ride-sharing applications.

They guarantee the consistency of transactions (important, for example, to avoid overbooking a place in a car or to track payments). PostgreSQL in particular offers geospatial extensions (PostGIS) very useful for querying location data – we could for example perform a query to find all the journeys whose starting point is within a radius of X km from a user. BlaBlaCar, for example, has operated for a long time on SQL (and probably still uses SQL for some of its structured data) before to add other tools Uber started on PostgreSQL before migrating to MySQL to some data which proves that these technologies can support growth significant.

Concretely, we will be able to model in PostgreSQL the Ko'Ride entities: Users, Profiles, Vehicles, Routes (carpooling suggestions with departure/arrival city, times, available spaces, etc.), Reservations (when a passenger reserves a seat on a trip), Payments, Reviews/Ratings, Messages (although for the chat we can also use a real-time NoSQL solution as mentioned). A database Well-normalized SQL works well for this linked data. If traffic increases, we will be able to vertically scale up the PostgreSQL server or move to a cluster, and possibly consider sharding (by geographic region for example) or moving to a managed cloud service (Cloud SQL, Amazon RDS) which facilitates scalability. Uber has shown that you can go quite far with a single relational base before having to segment it

1

2 .

- **Caches and other specialized databases :** To improve performance, it would be wise to to integrate a cache system as soon as usage increases. For example, a **Redis** in-memory cache can store the results of frequent queries (list of popular journeys, configuration of the app, etc.) and handle things like invalidated user sessions or JWT tokens, with very low latency Redis can also¹³ serve as a **Queue** (it has pub/sub) to propagate events between microservices later. In the longer term, if some features require different storage: e.g., location history GPS or activity logs could be better managed in a NoSQL database (Cassandra, MongoDB) to handle a massive volume of writes. **However, for Ko'Ride's MVP, This is not essential.** Good SQL optimization + a cache will suffice. The architecture must simply keep these expansion possibilities in mind.

- **Third-party integrations :** A ride-sharing app doesn't exist in a vacuum – it will take consume several external APIs, which sometimes influence the technological choice:

- **Mapping and Geolocation API:** Using **Google Maps API** is practically essential for obtaining the GPS coordinates of entered addresses, tracing routes road and possibly calculate estimated distances/times

24 25

. This involves integrating

Google Maps SDK on the mobile side (via Flutter/RN plugins) and call the Directions/Geocoding API on the backend side to calculate optimal routes or the central meeting point. One could also evaluate **Mapbox** (open source) to reduce costs, but Google Maps offers reliability and local knowledge (up-to-date Indonesian maps).

- **Payment Services:** Ko'Ride will need to integrate an online payment system to facilitate the transaction between passenger and driver (unless they go for an all-cash model, which is impractical). For an MVP, integrating an aggregator like **Stripe** or **PayPal** can work.

²⁴, but in Indonesia it will probably be necessary to take charge of local solutions (e-wallets like GoPay, OVO, DANA, or local bank transfers). One option is to use an Indonesian provider (such as Midtrans, DOKU) that offers unified APIs for several local payment methods. Technically, this means that the backend will have to communicate securely with these APIs (via REST/HTTPS), manage payment confirmation webhooks, etc. Both Spring Boot and NestJS have libraries to easily consume external services (HTTP client, etc.). **Security** will be paramount in this part – API keys will have to be stored properly, and possibly use a proxy server if regulations require it.

- **Sending SMS or emails:** For phone number verification (often a necessary step to establish trust, as Uber/Grab do), you can use a service like **Twilio** to send OTP SMS messages. Again, a simple API request from the backend will trigger the sending. You can also consider sending notification emails (booking confirmation, payment receipt) via a service like SendGrid. These choices do not greatly affect the stack, except that it is easier to use them in a language where many SDKs exist (fortunately, Twilio/SendGrid provide SDKs in JS, Java, etc. so Node or Java will be able to use them).
- **Social authentication:** Optionally, allowing users to log in via Google/Facebook can be a plus. Here, using Firebase Authentication could simplify the implementation (it handles Google/FB OAuth, etc.), or one can use specific SDKs (Passport.js on Node, or Spring Social in Java for example). This choice can be made depending on the available time, but is not essential for the MVP.

In summary, for the Ko'Ride backend, we recommend a **RESTful API built with NestJS (Node.js & TypeScript)**, connected to a main **PostgreSQL** database, and enriched with caches (**Redis** eventually). This API will manage the central business logic. It will rely on the appropriate **cloud services** for cross-functional features: sending push notifications (FCM), SMS (Twilio), secure payment, and use of the Google Maps API for everything geographical. This backend stack is consistent with the general recommendations for a carpooling app: a reference article suggests Node.js/Express or Python/Django as backend languages for this type of project, coupled with PostgreSQL or MongoDB as needed – here we choose Node/SQL for the reasons explained. Emphasis should be placed on a **modular architecture** of the code (by business domains) to facilitate future evolution.

²⁶ ²⁷

Infrastructure, deployment and scalability

Beyond language choices, the architecture must be designed to move from prototype to large-scale production gradually, without completely redesigning the stack. Here's an architectural roadmap for Ko'Ride:

- **MVP – simple and economical architecture:** For the functional prototype and the first months of commissioning, the application can be deployed in a very simple way. For example, host the backend on a single cloud server (a VM or a container in the cloud). A pragmatic choice would be to use a PaaS (*Platform as a Service*) like **Heroku** or **Railway** for

Deploy the Node API quickly without managing infrastructure, or a Docker container on an Amazon EC2/Google Compute Engine VM. The initial cost is low (a few dozen euros per month maximum). The PostgreSQL database can be initially hosted on a managed service (Amazon RDS, Google Cloud SQL) to gain reliability (automatic backups, etc.) without administrative effort – these services have free or low-cost tiers sufficient to get started. The Angular application can be hosted statically (on Firebase Hosting, Vercel, Netlify or even an S3 bucket behind CloudFront) to be served to web users.

The Flutter/RN mobile app will be distributed via stores (Google Play and Apple App Store). At this stage, **all components could be monolithic** : a single backend service manages everything, connected to a single database. This simplicity reduces the risk of inter-service bugs and costs. We still make sure from the start to separate the *environments* (test database vs. production database, etc.), and to set up basic continuous integration (e.g. GitHub Actions to launch tests and automatically deploy to the PaaS).

- **Gradual scaling:** If the application gains users (for example, starts to have thousands of active users each day), it will be necessary to ensure that it can handle the load. The Node.js backend can be easily scaled horizontally: you can run **multiple Node instances** in parallel (on multiple containers or machines) behind a **load balancer**. Cloud services like Heroku do this automatically (dynos), or you can use Docker/Kubernetes on AWS/GCP to orchestrate multiple instances. The stateless REST architecture makes this easy (all instances can serve requests interchangeably). For this to work, user sessions will need to be either stateless (idempotent via JWT tokens stored client-side) or shared via central storage (e.g., Redis to store sessions if necessary). The database can be scaled higher (upgrade to a higher tier of the managed database). We will probably introduce a **Redis cache system** at this stage to absorb frequent readings without constantly requesting PostgreSQL, and possibly set up a CDN for static content (profile images, files) by hosting them on object storage such as Amazon S3 or Google Cloud Storage.

- **Security and reliability in production:** As we grow, aspects like security (protection against injections, XSS, API call security, encrypting SSL communications – possibly via an NGINX proxy on input), supervision (logs, monitoring) become crucial. It will be necessary to use a monitoring service (Datadog, NewRelic, or the open-source Prometheus/Grafana) to track performance, error tracing (Sentry for example) to spot bugs in production. The infrastructure will also need to be redundant: having **several** geographical areas if we cover all of Indonesia, in order to reduce latency for users on different islands for example. Large players like Uber and Grab have very sophisticated multi-region deployments (Uber mentions an active-active deployment on several data centers with automatic failover in case of failure), but at our scale we can rely on cloud regions (for example deploying on a data center in Southeast Asia, like Jakarta or Singapore, to be close to users). BlaBlaCar, when they migrated to Google Cloud, made this choice precisely to be able to easily test on new markets at scale without heavy investment in servers.

28

29

- **Evolution towards modular microservices:** Eventually, if Ko'Ride explodes in popularity in the Indonesian archipelago, the architecture will need to evolve to support a massive user base and new features. We can then apply a gradual transformation into **microservices**. The idea would be to identify areas of the system that can be isolated into independent services – for example, a service dedicated to notifications, a service dedicated to the trip recommendation/matching engine, a payment service, etc. Each of these microservices could be developed/deployed separately, possibly with different technologies if

justified (this is what the giants did: Uber has microservices in Go for real time, in Python for machine learning, etc.

³⁰). Communication between microservices would be done via internal REST APIs or via messages (Kafka, RabbitMQ) to further decouple.

However, it is important to emphasize that it is neither necessary nor desirable to rush into microservices too early.

It will first be necessary to have reached a level of complexity and team size that justifies this division.

⁴ In the meantime, we can draw inspiration from “hexagonal” or modular principles in the monolithic code to prepare the ground – for example, by structuring the NestJS repo into modules per subdomain (User module, Journey module, Payment module, etc.), each with its own service and data model, so that they can be extracted more easily later. BlaBlaCar, for example, indicates that it uses the hexagonal architecture in its codebases to maintain flexibility. This way, when the time comes, extracting a module from the monolith to an independent microservice will be easier, because its dependencies will be clear and limited. ³¹

- **Real-time scalability** : A particular challenge as we evolve will be the scalability of real-time features (location tracking, dispatch). At a small scale, a Node server with Socket.io can handle this, but at very large scale (tens of thousands of simultaneous connections), solutions such as **dedicated WebSockets servers** will need to be considered (e.g., deploying a Socket.io cluster with a shared store, or using managed services like Pusher, PubNub if it becomes too complex internally). Uber, for example, has developed in-house systems (Ringpop, etc.) for high availability in real time and uses Kafka extensively to distribute events. Ko'Ride will not be there at the beginning, but the use of modern cloud tools (like Google Pub/Sub or AWS SNS/SQS) can facilitate the transition to an event-driven architecture when necessary, without having to ³² rewrite everything.

In summary, the proposed architecture is **progressive** : we start with a simple, centralized deployment (Node.js + PostgreSQL monolith) that covers all the needs of the MVP. This base must be developed following good separation of concerns practices to avoid a spaghetti monolith. As the application grows, we **scale this monolith horizontally** (more instances, cache, CDN) and then **cut into services** only when the volume and the organization of the team require it. This iterative approach has proven itself – “almost all microservices success stories started with a monolith that was then cut”.

³³ Ko'Ride will therefore be able to draw inspiration from this philosophy, keeping in mind the performance and cost objectives: using the cloud to avoid the initial investment (we pay per use), monitoring expenses (the choice of open-source technologies avoids licenses, and a lightweight stack like Node consumes few resources initially), and optimizing gradually.

Conclusion: Recommended Stack for Ko'Ride

In summary, here is the **recommended technology stack** for developing Ko'Ride in the given context:

- **Web Frontend**: Angular (TypeScript) – for a rich web application, taking advantage of the team's expertise. The UI will be designed by Léo (UX/UI) in collaboration with the devs, ensuring ergonomics for the Indonesian audience (language, simplicity, trust). Angular will provide a solid foundation for the front-end code, and will eventually allow for the reuse of pieces in a PWA.
- **Mobile App**: Flutter (Dart) cross-platform framework – to target Android and iOS with a single codebase. Flutter offers development speed and near-native performance, as well as design consistency across platforms. Alternatively, React Native

(JavaScript/TypeScript) is a valid choice if the team prefers, as it integrates well with a Node backend and benefits from a broad ecosystem. In both cases, we will have the features crucial mobile functions (GPS, notifications) managed via plugins.

- **Backend/API:** Node.js with the NestJS (TypeScript) framework. This will provide a REST API centralized, following a modular MVC architecture. NestJS provides an inspired structure of Angular which will be familiar, and runs on Node which is efficient in I/O and adapted to the needs real time. Secure REST endpoints (JWT auth) can be defined for all features. The backend will be able to use Socket.io for real-time, and rely on Firebase FCM to push mobile notifications.
- **Database:** PostgreSQL as the main (relational) database. It will store reliably structured information (users, journeys, reservations, payments, opinion...). PostgreSQL offers consistency and supports geospatial queries, useful for the carpooling. We plan to optimize with geographic indexes, and to use a **Redis cache** to speed up frequent reads and manage sessions if necessary.
- **Cloud Infrastructure:** Deployment on a major cloud provider with presence in Asia Southeast (e.g. **AWS** or **Google Cloud**). We will use managed services to save money. time: EC2 instance or Kubernetes service for the backend, managed database (Amazon RDS or Cloud SQL) for PostgreSQL, Cloud Storage (S3 bucket) to store photos profile or documents, CDN to serve static website content quickly to users in Indonesia. BlaBlaCar, for example, migrated to Google Cloud to support its global growth ⁹, which shows the interest of a scalable cloud infrastructure. Ko'Ride will only have to "turn up the volume" on these services in the event of a traffic peak, without changing the stack.
- **External integrations: Maps/**
- **Geolocation:** Google Maps API for geocoding (translate addresses into coordinates), route calculation and map display. Integrated Google Maps SDK on the front (web and mobile).
- **Payment:** Integration of a Stripe-type service for the MVP (card payment) and preparation to integrate local wallets. The goal is frictionless, secure in-app payment (PCI DSS compliant).
- **Push & SMS Notifications:** Firebase Cloud Messaging for mobile push notifications SMS service (Twilio or local ²⁰ equivalent) for phone verification and alerts if needed.
- **Auth and Security:** JWT management for authentication via the backend. Ability to use Firebase Auth or Auth0 if you want to speed up the implementation of registration/login in a secure, but a NestJS JWT module coupled with PostgreSQL (to store fingerprints of passwords, etc.) may be sufficient. End-to-end SSL encryption via HTTPS (a TLS certificate on the API domain name, managed by an Nginx proxy or the cloud load balancer).

This stack is consistent with modern application development **practices**. carpooling. It prioritizes productivity (Angular/NestJS/Flutter allow rapid development with a maintainable code base) while aligning with what is found in the stacks recommended for this type of application (for example, a specialized blog lists Flutter, Node.js, PostgreSQL, Firebase as an effective combo for carpooling apps). Above all, it is **suitable for the junior team**: use of known technologies (JavaScript/TypeScript), large support ecosystem (very active Angular, Node, Flutter communities), and the possibility of attracting local developers who master these tools.

Finally, the proposed architecture is **realistic and progressive**: it allows the release of a functional MVP quickly without sacrificing scalability. As Ko'Ride grows, the team will be able to build on the solid foundations laid (code modularity, scalable cloud, good coding practices) to add new features (e.g. a machine-optimized ride-sharing suggestion algorithm learning, which could be developed in Python as a separate microservice if needed in the future). By following

Following the example of the big players in the sector but on a human scale, Ko'Ride will be able to evolve from a small local application to, we hope, a major carpooling platform in Indonesia, all with a manageable and efficient stack.

Sources: The above recommendations are based on case studies of similar applications and feedback from large companies in the field. For example, Uber has shown the effectiveness of a simple start (Python/Postgres monolith) before moving to microservices to support its growth. Lyft uses a Python/Flask stack in microservices with a JS front end, illustrating that a classic web stack can run a mobility platform. Carpooling app development guides confirm the orientation towards Node.js/Express or Django as the backend, and Flutter/React Native on the mobile front, with PostgreSQL as a reliable base. Finally, BlaBlaCar and Grab have focused on the cloud and microservices for scale, which remains a long-term objective for Ko'Ride once the product is validated. By synthesizing these international best practices and adapting them to the Ko'Ride context, we obtain the optimal technology stack to combine **development speed now and serene growth later**.

9

3

2

6

1 Up: Portable Microservices Ready for the Cloud | Uber Blog <https://www.uber.com/blog/up-portable-microservices-ready-for-the-cloud/>

2 12 13 28 The Uber Engineering Tech Stack, Part I: The Foundation | Uber Blog <https://www.uber.com/blog/tech-stack-part-one-foundation/>

3 How BlaBlaCar Works: Revenue Model Explained - IdeaUsher <https://ideausher.com/blog/blabla-car-business-model-how-it-works/>

4 33 Starting With a Monolith or Microservices: How New Technology Is Changing the Conventional View <https://blog.dreamfactory.com/starting-with-a-monolith-or-microservices-how-new-technology-is-changing-the-conventional-view>

5 6 30 32 The Uber Engineering Tech Stack, Part II: The Edge and Beyond | Uber Blog <https://www.uber.com/blog/uber-tech-stack-part-two/>

7 What is the software stack of Lyft? - Quora <https://www.quora.com/What-is-the-software-stack-of-Lyft>

8 TIA x Grab - Tech in Asia <https://www.techinasia.com/tag/tia-x-grab>

9 14 29 BlaBlaCar Case Study | Google Cloud <https://cloud.google.com/customers/blabla-car>

10 20 22 25 How To Develop A Carpooling App: Features, Steps and Cost <https://emizentech.com/blog/develop-a-car-pooling-app.html>

11 Carpooling App Development: Steps, Features, and Cost Breakdown | Codementor <https://www.codementor.io/@rajdeep885527/carpooling-app-development-steps-features-and-cost-breakdown-2qw1ba5o1b>

15 16 17 18 19 23 Analysis of all docs.odt <file:///file-Cm77XvCyFhHCZ5FExgtpEE>

21 nestjs #nodejs #java #springboot #comparison #developers - LinkedIn https://www.linkedin.com/posts/piyushcodes6_nestjs-nodejs-java-activity-7229929778814824450-V9hW

24 26 27 Building an App Like BlaBlaCar: A Comprehensive Guide <https://www.linkedin.com/pulse/building-app-like-blabla-car-comprehensive-guide-vibidsoft-reigf>

³¹ Pragmatic Hexagonal Architecture - BlaBlaCar - Medium <https://medium.com/blablacar/pragmatic-hexagonal-architecture-f0dbc70b736d>